Design of a Collision-Resilient Hash-Based Caching System for High-Throughput Product Lookups in E-Commerce

Submitted in partial fulfilment for the course

of

CSA0315 – DATA STRUCTURES

towards the award of the degree of

BACHELOR OF ENGINEERING

IN

AI&ML

Submitted by

HARSHITH G

(192525394)

Under the Supervision of

Dr. T. Rajesh Kumar

SAVEETHA SCHOOL OF ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Artificial Intelligence and Machine Learning
Programme:	B.Tech
Course Code & Course Name	CSA0315 – Data Structures
Academic Year / Batch	2nd Year / 2025
Faculty Name	Dr. T. Rajesh Kumar
Assignment Title	Design of a Collision-Resilient Hash-Based Caching System for High-Throughput Product Lookups in E-Commerce
Date of Issue	02/09/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Apply hashing concepts and collision-resolution techniques to design efficient search/lookup services
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG9 – Industry, Innovation and Infrastructure; SDG 8 – Decent Work and Economic Growth
Industry / Societal Relevance	Product-catalog search caching, database indexing, session/token stores. Compiler symbol tables, spell-checkers

1. Problem Understanding and Formulation

1.1 Introduction

Modern e-commerce platforms handle a very large number of product-search and product-lookup requests every day. Customers expect product information such as product name, price, category, and availability to be retrieved quickly. If every product lookup directly accesses the backend database, the database may become overloaded, especially during high-traffic events such as flash sales and promotional campaigns.

To reduce database access time and improve system performance, an in-memory caching system can be placed between the product-search service and the database. A hash table is suitable for this purpose because it provides approximately constant average-time complexity for insertion, searching, and updating.

The proposed system uses product IDs such as SKU12345 as keys. A hash function converts each product ID into an integer index corresponding to a position in a fixed-size array. However, different product IDs may produce the same index. This condition is known as a collision.

Therefore, the main objective of this project is to design a collision-resilient hash-based cache capable of efficiently handling product lookups even when collisions occur frequently.

The assignment requires comparison of two collision-resolution techniques: separate chaining and open addressing, using the same dataset and hash function.

1.2 Problem Statement

The proposed system addresses the problem of efficiently storing and retrieving product information in an e-commerce application using a hash-based cache.

The major challenges are:

1. Large numbers of product lookup requests must be processed efficiently.
2. Product IDs may not be uniformly distributed.
3. Multiple product IDs may map to the same hash-table index.
4. Collisions must be resolved without losing existing data.
5. The cache must support insertion, searching, updating, and deletion.
6. The system must monitor its load factor.
7. Rehashing must automatically occur when the load factor exceeds 0.7.
8. The system must provide collision statistics.
9. Open addressing must correctly handle deleted entries using tombstones.
10. The hash function should provide good distribution and reduce clustering.
11. The implementation must demonstrate the underlying hash-table mechanism rather than relying on a built-in dictionary or HashMap.

1.3 Expected Outcomes

The expected outcomes of the proposed system are:

- Fast product insertion and lookup.
- Efficient collision handling.
- Correct deletion and updating of products.
- Automatic rehashing when required.
- Measurement of collision and probing statistics.
- Comparison between separate chaining and open addressing.
- Improved cache performance under high request loads.
- Reliable retrieval of all inserted products after rehashing.
- Reduced unnecessary database access.

1.4 Available Information

The cache stores information in the following general format:

Field	Description
Product ID	Unique identifier such as SKU12345
Product Name	Name of the product
Category	Product category
Price	Product price
Availability	Stock/availability information
Hash Index	Position calculated using hash function

Example:

Product ID	Product Name	Category	Price
SKU1001	Wireless Mouse	Electronics	₹799
SKU1002	Keyboard	Electronics	₹1299
SKU2001	Running Shoes	Footwear	₹2499
SKU3001	Backpack	Bags	₹1599

Synthetic product data is used so that no real customer or personal information is exposed, which is consistent with the assignment's privacy and ethical requirements.

1.5 Assumptions

The following assumptions are made:

- Every product has a unique product ID.
- The cache initially has a fixed capacity.
- Product IDs are strings.
- The hash function converts a string into an integer.
- The hash-table capacity is preferably selected as a prime number.
- The cache stores synthetic product information.
- Average hash-table operations should approach O(1).
- Rehashing is triggered when the load factor becomes greater than 0.7.

1.6 Constraints

The major constraints are:

- A fixed-size array-backed hash table must be used.
- Built-in HashMap, dictionary, or equivalent structures cannot be used as the core implementation.
- Both separate chaining and open addressing must be considered.
- Collision handling must be explicitly implemented.
- Rehashing must be implemented.
- Deleted slots in open addressing must use tombstones.
- The implementation should run on a standard student computer.
- The solution should use freely available tools.

2. Application of Course / Domain Knowledge

The project directly applies concepts from the Data Structures course, particularly hashing and collision-resolution techniques.

The assignment identifies the relevant course outcome as CO4: Apply hashing concepts and collision-resolution techniques to design efficient search/lookup structures.

2.1 Hashing

Hashing is a technique that converts a key into an array index using a hash function.

For a table of size m:

$$index = h(key) \bmod m$$

For example, if:

$$h("SKU1001") = 56$$

and:

$$m = 11$$

then:

$$index = 56 \bmod 11 = 1$$

Therefore, SKU1001 is initially placed at index 1.

2.2 Hash Function

A string-based hash function can process each character of the product ID.

One suitable polynomial hash is:

$$h(s) = \left(\sum_{i=0}^{n-1} s_i \times p^i \right) \bmod m$$

where:

- s_i= character value
- p= chosen multiplier
- m= table size

A practical implementation can use a rolling calculation:

$$hash = (hash \times p + character) \bmod m$$

This provides better distribution than simply adding character values.

2.3 Collision

A collision occurs when two different keys generate the same table index.

For example:

SKU1001 → index 5

SKU2035 → index 5

Both keys want to occupy index 5.

The system must therefore use a collision-resolution mechanism.

2.4 Separate Chaining

In separate chaining, each hash-table position contains a linked structure containing all entries mapped to that index.

Example:

Index 5

|

v

[SKU1001 | Mouse]

|

v

[SKU2035 | Keyboard]

|

v

[SKU3050 | Headphones]

Advantages:

- Simple collision handling.
- Multiple elements can occupy the same index.
- Deletion is relatively easy.
- Performance remains reasonable when collisions are moderate.

Disadvantages:

- Requires additional memory for linked nodes.
- Poor hash distribution can create long chains.
- Cache locality is lower compared with contiguous array storage.

2.5 Open Addressing

In open addressing, all elements are stored directly inside the hash-table array.

For this project, linear probing can be used.

The probing formula is:

$$index_i = (h(key) + i) \bmod m$$

where $i = 0,1,2, ...$

Example:

Initial index = 5

Index 5 → occupied

Index 6 → occupied

Index 7 → empty

Therefore:

SKU2035 → index 7

2.6 Primary Clustering

Linear probing can create primary clustering.

For example:

[5] [6] [7] [8] [9]

X X X X X

Several elements form a continuous block.

When another collision occurs, the system may need to traverse the entire block, increasing search time.

This is why the assignment specifically requires testing a cluster of six IDs under linear probing.

2.7 Load Factor

The load factor indicates how full the hash table is.

$$\alpha = \frac{n}{m}$$

where:

- n= number of stored elements
- m= table capacity.

For example:

$$n = 8,\ m = 11$$

$$\alpha = \frac{8}{11} = 0.727$$

Since:

$$0.727 > 0.7$$

rehashing must be triggered.

The assignment explicitly specifies 0.7 as the maximum load-factor threshold.

2.8 Rehashing

When the load factor exceeds 0.7, a larger table is created.

For example:

Old capacity = 11

↓

Load factor > 0.7

↓

Create larger table

↓

Recalculate hash positions

↓

Insert all existing entries

↓

Replace old table

Rehashing increases the table capacity and reduces the probability of collisions.

Although rehashing requires O(n) work at that particular point, insertion remains approximately O(1) amortized over many operations.

2.9 Tombstones

Deleting an element directly from an open-addressed table can break the probing sequence.

Example:

Index: 5 6 7

A B C

If B is deleted:

Index: 5 6 7

A X C

If index 6 is treated as completely empty, searching for C may stop at index 6 and incorrectly report a cache miss.

Therefore, a special tombstone marker is used.

EMPTY

OCCUPIED

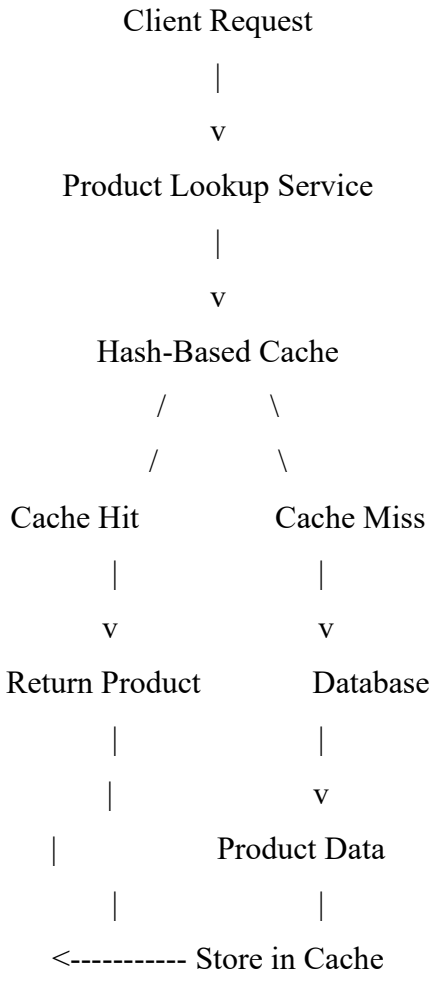
DELETED

This requirement is specifically included in the assignment.

3. Solution Methodology / Design / Implementation

3.1 Proposed System Architecture

The proposed caching architecture is:



The cache attempts to satisfy requests before accessing the database.

3.2 System Components

The system contains the following components:

1. Product Entry

Stores:

- Product ID
- Product Data
- Status

2. Hash Function

Converts the product ID into an array index.

3. Hash Table

Stores product entries.

4. Collision Resolver

Implements:

- Separate chaining
- Linear probing

5. Load Factor Monitor

Calculates:

$$\alpha = n/m$$

6. Rehashing Module

Creates a larger table when:

$$\alpha > 0.7$$

7. Statistics Module

Tracks:

- Number of collisions
- Longest chain
- Longest probe sequence
- Number of elements
- Table capacity
- Load factor

3.3 Functional Operations

The proposed system supports the required operations:

Insert

insert(productId, data)

Adds a new product or updates an existing product.

Search

search(productId)

Returns product information if present.

Delete

delete(productId)

Removes the specified product.

Load Factor

computeLoadFactor()

Calculates current table utilization.

Rehash

rehash()

Expands the table and reinserts existing entries.

Collision Statistics

getCollisionStats()

Reports collision and probe/chain statistics.

These operations directly correspond to the required functional operations in the assignment.

3.4 Insertion Algorithm – Separate Chaining

INSERT(key, data)

1. Calculate hash index.
2. Access the chain at that index.
3. Search the chain for the key.
4. If key exists:
    Update data.
5. Otherwise:
    Create a new node.
    Add node to the chain.
6. Increase element count.
7. Calculate load factor.
8. If load factor > 0.7:
    Rehash.

3.5 Search Algorithm – Separate Chaining

SEARCH(key)

1. Calculate hash index.
2. Access corresponding chain.
3. Traverse the chain.
4. Compare each key.
5. If matching key is found:
    Return product data.
6. Otherwise:
    Return cache miss.

3.6 Insertion Algorithm – Linear Probing

INSERT(key, data)

1. Calculate initial hash index.
2. Check the calculated position.
3. If position is empty:
   Insert product.
4. If position contains the same key:
   Update product.
5. Otherwise:
   Move to next index.
6. Continue using:
   (index + 1) % tableSize
7. Count each collision.
8. Insert when an available position is found.
9. Check load factor.
10. Rehash if load factor > 0.7.

3.7 Search Algorithm – Linear Probing

SEARCH(key)

1. Calculate initial index.
2. Check the index.
3. If key matches:
   Return product.
4. If slot is EMPTY:
   Return cache miss.
5. If slot is DELETED:
   Continue probing.
6. Otherwise:
   Move to next index.
7. Continue until key is found or the probe sequence ends.

3.8 Deletion Algorithm

For open addressing:

DELETE(key)

1. Calculate initial hash index.
2. Follow the probing sequence.
3. If key is found:
   Mark slot as DELETED.
4. Do not mark it as EMPTY.
5. Decrease element count.
6. Return successful deletion.

3.9 Rehashing Algorithm

REHASH()

1. Create a new table with larger capacity.
2. Select the next suitable prime capacity.
3. Reset collision statistics.
4. Traverse old table.
5. For every active entry:
   Recalculate hash index.
   Insert into new table.
6. Replace old table with new table.
7. Continue normal operations.

3.10 Example Hash Table

Assume:

Table size = 11

Example:

Product ID	Hash Value	Index
SKU1001	34	1
SKU1002	45	1
SKU2001	23	1
SKU3001	19	8

The first three keys generate the same index.

Separate Chaining

Index 1 → SKU1001 → SKU1002 → SKU2001

Index 8 → SKU3001

Linear Probing

Index 1 → SKU1001

Index 2 → SKU1002

Index 3 → SKU2001

Index 8 → SKU3001

This demonstrates how the two techniques resolve the same collision differently.

3.11 Complexity Analysis

Operation	Separate Chaining Average	Separate Chaining Worst	Open Addressing Average	Open Addressing Worst
Search	O(1)	O(n)	O(1)	O(n)
Insert	O(1)	O(n)	O(1)	O(n)
Delete	O(1)	O(n)	O(1)	O(n)
Rehash	O(n)	O(n)	O(n)	O(n)

4. Use of Appropriate Modern Tools / Techniques

The proposed Collision-Resilient Hash-Based Caching System is implemented as a menu-driven C program using Dev-C++. The implementation focuses on demonstrating the internal working of hashing and collision-resolution techniques rather than using built-in dictionary or hash-map libraries, as required by the assignment.

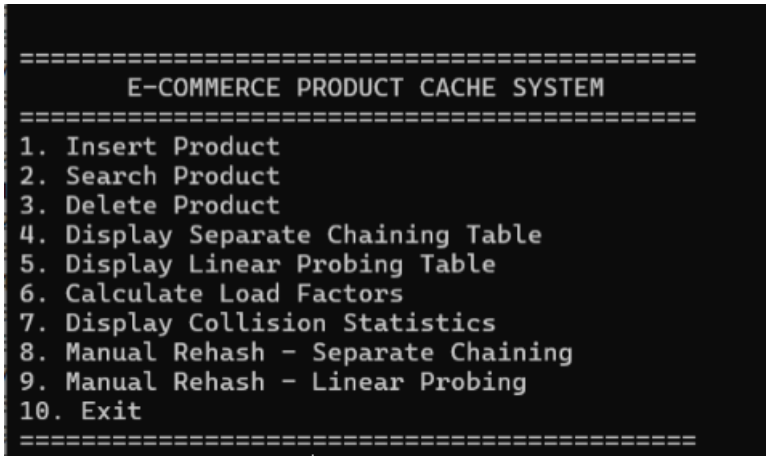
4.1 Development Tools Used

Tool / Technique	Purpose
C Programming Language	Implementation of the hash-based caching system
Dev-C++ IDE	Writing, compiling and executing the C program
Menu-Driven Interface	Allows the user to perform different cache operations interactively

Arrays	Used to implement the fixed-size hash table
Structures (struct)	Used to store product ID and product details
Pointers	Used for linked-list implementation in separate chaining
String Handling	Used to process product IDs such as SKU1001
Modular Arithmetic	Used for calculating hash-table indices
Separate Chaining	Collision-resolution technique using linked lists
Linear Probing	Collision-resolution technique using open addressing
Tombstones	Used to handle deletion in open addressing
Load Factor Calculation	Used to monitor table utilization
Rehashing	Used when load factor exceeds 0.7
Collision Statistics	Used to measure collisions and probe/chain lengths

4.2 Menu-Driven Program

The program provides an interactive menu through which the user can perform the required hash-table operations.



This menu allows the user to test the system without modifying the source code for every operation.

4.3 C Data Structures Used

A struct can be used to represent a product:

```
typedef struct Product
{
    char productID[MAX_ID];
    char productName[MAX_NAME];
    float price;
    int quantity;
} Product;
```

For separate chaining, pointers and linked-list nodes are used:

Hash Table

|

+--- Index 0 → NULL

|

+--- Index 1 → Product → Product → NULL

|

+--- Index 2 → Product → NULL

|

+--- Index 3 → NULL

For open addressing, products are stored directly in the array.

4.4 Hashing Technique

The program implements its own hash function instead of using a built-in hashing library.

The product ID is converted into a numerical hash value and mapped to an array index:

$$index = hash(key) \text{ mod } tableSize$$

For example:

Product ID: SKU1001

↓

Hash Function

↓

Hash Value

↓

Modulo Table Size

↓

Array Index

4.5 Collision-Resolution Techniques

Two collision-resolution techniques are implemented in the C program:

1. Separate Chaining

Multiple products that produce the same index are stored in a linked list.

2. Linear Probing

If the calculated position is occupied, the program checks the next available position using:

$$index = (index + 1) \bmod tableSize$$

This allows both techniques to be tested and compared using the same product dataset, as required by the assignment.

4.6 Load Factor and Rehashing

The C program calculates the load factor using:

$$Load\ Factor = \frac{Number\ of\ Elements}{Table\ Size}$$

When:

$$Load\ Factor > 0.7$$

the program triggers rehashing.

For example:

Number of elements = 8

Table size = 11

Load Factor = 8/11

= 0.727

Since 0.727 > 0.7, the table is resized and the existing elements are reinserted into the new table. This follows the assignment's specified rehashing requirement.

4.7 Testing and Debugging Using Dev-C++

Dev-C++ is used to:

1. Write the C source code.
2. Compile the program.
3. Identify syntax and compilation errors.
4. Execute the menu-driven program.
5. Test insertion, searching and deletion.

6. Verify collision handling.
7. Test load-factor calculation.
8. Verify automatic rehashing.
9. Test tombstone handling.
10. Record console outputs and screenshots.

The assignment recommends providing execution evidence such as code listings, screenshots and execution logs.

5. Results, Testing and Validation

The system should be tested using the six required test cases specified in the assignment.

A) INSERTION

```
==========================================
       E-COMMERCE PRODUCT CACHE SYSTEM
==========================================
1. Insert Product
2. Search Product
3. Delete Product
4. Display Separate Chaining Table
5. Display Linear Probing Table
6. Calculate Load Factors
7. Display Collision Statistics
8. Manual Rehash - Separate Chaining
9. Manual Rehash - Linear Probing
10. Exit
==========================================
Enter your choice: 1

Select Collision Resolution Method
1. Separate Chaining
2. Linear Probing
Enter method: 1

Enter Product ID: 2
Enter Product Name: Mug
Enter Price: 25
Enter Quantity: 3
```

B) DISPLAY

```
==========================================
       E-COMMERCE PRODUCT CACHE SYSTEM
==========================================
1. Insert Product
2. Search Product
3. Delete Product
4. Display Separate Chaining Table
5. Display Linear Probing Table
6. Calculate Load Factors
7. Display Collision Statistics
8. Manual Rehash - Separate Chaining
9. Manual Rehash - Linear Probing
10. Exit
==========================================
Enter your choice: 4

======================================
       SEPARATE CHAINING TABLE
======================================
[0] -> NULL
[1] -> NULL
[2] -> NULL
[3] -> NULL
[4] -> NULL
[5] -> 1
[6] -> 2
[7] -> 3
[8] -> 4
[9] -> NULL
[10] -> NULL
```

C) DELETE

```
==========================================
       E-COMMERCE PRODUCT CACHE SYSTEM
==========================================
1. Insert Product
2. Search Product
3. Delete Product
4. Display Separate Chaining Table
5. Display Linear Probing Table
6. Calculate Load Factors
7. Display Collision Statistics
8. Manual Rehash - Separate Chaining
9. Manual Rehash - Linear Probing
10. Exit
==========================================
Enter your choice: 3

Select Delete Method
1. Separate Chaining
2. Linear Probing
Enter method: 1

Enter Product ID to delete: 2

Product deleted successfully.
```

D) COLLISION STATISTICS

```
========================================
       SEPARATE CHAINING TABLE
========================================
[0] -> SKU2001
[1] -> NULL
[2] -> NULL
[3] -> NULL
[4] -> NULL
[5] -> 1
[6] -> 11
[7] -> 3
[8] -> SKU1001 -> 4
[9] -> NULL
[10] -> NULL

Number of Products : 6
Table Size         : 11
Load Factor        : 0.545
Collisions         : 1
Longest Chain      : 2
```

E) MANUAL HASHING

```
Enter your choice: 8

========================================
       REHASHING CHAINING TABLE
========================================
Old Size : 11
New Size : 23
Rehashing completed successfully.
```

5.1 Performance Metrics

The following metrics should be collected:

Metric	Purpose
Number of insertions	Measure workload
Number of searches	Measure lookup workload
Collision count	Evaluate hash distribution
Longest chain	Evaluate chaining
Longest probe	Evaluate open addressing
Load factor	Monitor table utilization
Rehash count	Measure resizing overhead
Successful searches	Validate correctness
Cache misses	Validate negative searches

6. Analysis, Trade-offs and Justification

6.1 Separate Chaining vs Linear Probing

Feature	Separate Chaining	Linear Probing
Storage	Array + chains	Array only
Collision handling	Linked nodes	Sequential probing
Memory	Higher	Lower
Cache locality	Lower	Higher
Deletion	Easy	Requires tombstones
Clustering	Less affected	Primary clustering
Implementation	Moderate	Moderate
High load factor	More tolerant	Performance degrades faster
Memory allocation	More	Less
Lookup	Efficient	Efficient at low/moderate load

6.2 Advantages of Separate Chaining

- Simple collision management.
- Multiple keys can share one index.
- Deletion does not require tombstones.
- Less sensitive to table saturation.
- Suitable when collisions are frequent.

6.3 Advantages of Linear Probing

- Uses contiguous memory.
- Good cache locality.
- No linked-node allocation.
- Lower memory overhead.
- Simple implementation.

6.4 Disadvantages of Linear Probing

The main limitation is primary clustering.

If many product IDs map to nearby positions, the probe sequence becomes longer.

For a high-throughput system, excessive probing can increase lookup latency.

6.5 Recommended Design

For this project, separate chaining can be selected as the more collision-resilient primary strategy, because the problem explicitly states that product IDs may not be uniformly distributed and collisions are expected to be frequent.

However, linear probing should still be implemented and measured because the assignment specifically requires comparing both techniques.

The final decision should be based on measured:

- collision count,
- chain length,
- probe length,
- memory usage,
- lookup behaviour,
- rehash frequency.

This makes the engineering decision evidence-based rather than based only on theoretical assumptions.

6.6 Hash-Flooding Consideration

A poorly designed hash function may cause many product IDs to map to the same location.

For example:

SKU-A001 → 4

SKU-A002 → 4

SKU-A003 → 4

SKU-A004 → 4

...

This can cause excessive chains or long probe sequences.

Therefore, the hash function should:

- Process the complete product ID.
- Provide good distribution.
- Avoid simple character-sum hashing.
- Use modular arithmetic.
- Be tested using different SKU patterns.
- Monitor collision statistics.

The assignment specifically requires the distribution quality of the hash function to be justified because the cache should be defensible against poorly chosen or adversarial keys.

7. Broader Considerations / Professional Responsibility

7.1 Security

The cache should avoid exposing sensitive product or customer information through logs.
Only synthetic product IDs and test data should be used during development.

7.2 Privacy

No personal customer information is required for this project.
Test data should consist of synthetic product records.

7.3 Reliability

The system must ensure that:

- Products are not silently lost.
- Existing entries are not overwritten incorrectly.
- Deleted entries do not break search.
- Rehashing preserves all valid entries.
- Wrap-around probing works correctly.

These requirements correspond to the reliability and safety requirements specified in the assignment.

7.4 Sustainability

An efficient hash table reduces unnecessary database queries and computational overhead.
Maintaining a suitable load factor helps prevent excessively long searches and unnecessary CPU usage.
The design should also avoid unnecessary repeated resizing and memory allocation.

7.5 Economic Benefits

A high-performance cache can reduce the number of requests sent to the backend database.
This can:

- Reduce database workload.
- Improve response time.
- Support more simultaneous users.
- Reduce infrastructure requirements.
- Improve scalability during traffic spikes.

7.6 Professional Responsibility

A software engineer should ensure that the implementation is:

- Correct.
- Testable.
- Maintainable.
- Secure.
- Well documented.
- Reproducible.

All assumptions, limitations and performance results should be clearly documented.

8. Technical Documentation and Reflection

8.1 Implementation Documentation

The implementation should document:

1. Hash-function design.
2. Hash-table structure.
3. Collision-resolution methods.
4. Insert operation.
5. Search operation.
6. Delete operation.
7. Tombstone handling.
8. Load-factor calculation.
9. Rehashing mechanism.
10. Collision-statistics calculation.

The report should include suitable pseudocode, diagrams, trace tables and screenshots as evidence of implementation and testing. The assignment explicitly requests code listings, screenshots and execution evidence where appropriate.

8.2 Learning Reflection

Through this project, several practical concepts of data structures can be understood beyond theoretical classroom learning.

The major learning outcomes include:

- Understanding how hashing is applied to real-world systems.
- Learning how collisions affect performance.
- Understanding the difference between chaining and open addressing.
- Implementing hash functions for string-based keys.
- Understanding load-factor management.

- Implementing rehashing.
- Understanding why tombstones are required.
- Measuring actual performance instead of relying only on theoretical complexity.
- Understanding the trade-off between memory usage and lookup performance.
- Applying data-structure concepts to an e-commerce application.

8.3 Possible Improvements

If additional time and resources were available, the following improvements could be implemented:

1. Better Hash Functions

A stronger hash function could provide improved resistance against deliberate collision patterns.

2. Quadratic Probing

Instead of:

$$h(k) + i$$

quadratic probing can use:

$$h(k) + i^2$$

to reduce primary clustering.

3. Adaptive Resizing

The system could use different resizing thresholds depending on traffic patterns.

4. Cache Expiration

Products could automatically expire after a predefined period.

5. LRU Replacement

A Least Recently Used mechanism could remove old products when memory is limited.

6. Concurrent Access

Locks or concurrent data structures could be introduced to support multiple requests simultaneously.

7. Performance Dashboard

A dashboard could visualize:

- Lookup latency
- Collision count
- Load factor
- Probe length
- Rehash operations

9. Conclusion

The proposed Collision-Resilient Hash-Based Caching System for High-Throughput Product Lookups in E-Commerce demonstrates how hashing and collision-resolution techniques can be applied to solve a practical high-performance lookup problem.

The system uses a custom array-backed hash table with an explicitly implemented hash function. Two collision-resolution strategies, separate chaining and linear probing, are implemented and compared. Load-factor monitoring ensures that rehashing is triggered whenever the load factor exceeds 0.7, preventing the table from becoming excessively crowded.

The system also handles important edge cases such as cache misses, collisions, deletion and tombstones. Collision statistics and probe/chain lengths provide quantitative information for evaluating the two approaches.

Based on theoretical analysis, separate chaining provides better tolerance to frequent collisions, while linear probing provides better memory locality and lower storage overhead. The final selection should be based on the experimental collision and performance results obtained from the same dataset.

Overall, the project demonstrates the practical importance of hashing in high-throughput systems such as e-commerce product catalogs, database indexing and caching systems. The assignment therefore achieves its intended Data Structures course outcome of applying hashing and collision-resolution techniques to efficient search and lookup structures.

10. References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.
2. Mark Allen Weiss, Data Structures and Algorithm Analysis, Pearson.
3. Robert Lafore, Data Structures and Algorithms in Java, Sams Publishing.
4. Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, Fundamentals of Data Structures, Computer Science Press.
5. Department of Computer Science and Engineering, Data Structures Course Notes and Laboratory Materials, 2026–27.
6. Class Assignment – Design of a Collision-Resilient Hash-Based Lookup Cache for an E-Commerce Search Engine, Data Structures, B.Tech, Academic Year 2026–27.